Local Planning in an Unknown Environment: A Comparative Study of DWA and APF on TurtleBot3

Devang Bordoloi
Roll No: 2025122003
devang.bordoloi@research.iiit.ac.in

Sai Poojith Mididoddi
Roll No: 2025122010
mididoddi.poojith@research.iiit.ac.in

Mohammed Adeeb
Roll No: 2025122005
mohammed.adeeb@research.iiit.ac.in

Department of Electronics and Communication Engineering
INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY HYDERABAD

***Abstract*—Reactive local motion planning is a critical component of autonomous mobile robot navigation, enabling real-time obstacle avoidance in unknown environments without prior maps. This paper presents a systematic implementation and experimental comparison of two classical reactive local planning algorithms on a TurtleBot3 Burger platform operating under ROS2 in Gazebo simulation. The Dynamic Window Approach (DWA) samples the robot's reachable velocity space and selects trajectories via a multi-term cost function, while the Artificial Potential Field (APF) method drives the robot along the gradient of an attractive-repulsive force field augmented with tangential escape and gap-based recovery. Experimental trials on identical static and dynamic obstacle environments reveal a decisive performance gap: DWA reached the goal in 36.3 s over 6.28 m with zero recovery events, while APF required 388 s over 70.77 m with eleven gap-escape triggers. A dynamic obstacle extension further validated DWA's predictive advantage. Quantitative results, pseudocode, and a structured comparative analysis are provided, demonstrating that DWA's forward-looking trajectory evaluation makes it the superior algorithm for cluttered and dynamic environments.**

***Index Terms*—Local Motion Planning, Dynamic Window Approach, Artificial Potential Field, TurtleBot3, Collision Avoidance, ROS2, Gazebo Simulation, Dynamic Obstacles, Mobile Robotics.**

I. INTRODUCTION

Autonomous mobile robots operating in real-world environments must navigate safely without prior knowledge of the surroundings. The task of local motion planning—generating collision-free velocity commands from instantaneous sensor data—is therefore fundamental to any deployment in unknown or dynamic spaces [1]. While global planners such as A* or Dijkstra's algorithm compute optimal routes given a complete map, they cannot react to unmodelled or moving obstacles; local planners must fill this gap in real time. Two paradigms have dominated reactive local planning since the 1980s–90s. *Force-field methods*, introduced by Khatib [2], compute an instantaneous resultant of attractive and repulsive virtual forces and are computationally lightweight, making them attractive for resource-constrained platforms. *Trajectory sampling methods*, exemplified by the Dynamic Window Approach of Fox et al. [1], explicitly account for robot kinematics by evaluating forward-simulated candidate trajectories, at the cost of greater computational effort. Despite their long history, a rigorous quantitative comparison of these two approaches on a modern ROS2 platform, with pseudocode-level implementation transparency, dynamic obstacles, and real measured metrics, is absent from the recent literature. This work addresses that gap with the following contributions:

1) A complete ROS2 Humble Python implementation of DWA (`dwa_planner.py`) and APF (`apf_planner.py`), including three local-minima escape mechanisms in APF.
2) A dynamic obstacle manager node (`dynamic_obstacle_manager.py`) that spawns and moves three box obstacles in Gazebo Classic using the `/set_entity_state` service.
3) Controlled Gazebo experiments with identical environments and quantitative metrics: time to goal, path length, minimum obstacle clearance, and recovery event count.
4) A structured comparative analysis identifying the root-cause architectural differences that explain the observed performance gap.

II. RELATED WORK

The Artificial Potential Field method was introduced by Khatib [2] for real-time manipulator and mobile robot obstacle avoidance. Its fundamental limitation—convergence to local minima where attractive and repulsive forces cancel—was identified early and has motivated numerous extensions. Borenstein and Koren [3] proposed the Vector Field Histogram (VFH), which builds a polar obstacle density histogram and selects the widest free sector, avoiding the local-minimum problem more robustly. Ge and Cui [4] introduced a goal-dependent repulsive potential that prevents the robot from stopping near obstacles when close to the goal. The tangential force augmentation used in this work follows the direction proposed by Borenstein and Koren for cases of direct force opposition.

The Dynamic Window Approach was proposed by Fox, Burgard, and Thrun [1] as part of the RHINO and MINERVA museum-tour robots. It extended earlier curvature velocity methods [5] by bounding the search to kinematically reachable velocities. Subsequent work on the Timed Elastic Band (TEB) planner [6] generalised trajectory optimisation to include time parameterisation and kinodynamic constraints. The ROS Navigation Stack adopted DWA as its default local planner, contributing to its widespread adoption and practical validation.

This paper focuses on a direct ROS2 implementation comparison of the two foundational approaches—APF and DWA—rather than their more complex descendants, providing a clean baseline for understanding the trade-offs inherent in each paradigm.

III. System Setup

A. Platform and Environment

All experiments were conducted on a TurtleBot3 Burger differential-drive robot simulated in Gazebo Classic under ROS2 Humble. The Gazebo world contained cylindrical pillars, a rectangular box, and a bar obstacle forming a semi-enclosed region. Three additional box obstacles were animated as dynamic agents (see Section VI). The robot started at the world origin and was commanded to reach $(g_x, g_y) = (4.0, 0.0)$ m.

B. Sensing and Control Architecture

Both planners subscribe to:

- `/scan` — LDS-01 360° LIDAR (QoS: BEST_EFFORT, depth 10).
- `/odom` — wheel odometry for pose and velocity.

Both publish `/cmd_vel` (geometry_msgs/Twist) at **10 Hz** ($\Delta t = 0.1$ s). Platform limits enforced in both planners: $v_{\max} = 0.22$ m/s, $v_{\min} = -0.05$ m/s, $\omega_{\max} = 2.84$ rad/s, $a_{\text{lin}} = 2.5$ m/s^2, $a_{\text{ang}} = 30.0$ rad/s^2. Goal tolerance is 0.3 m Euclidean radius.

IV. Dynamic Window Approach

A. Overview

The Dynamic Window Approach [1] searches the space of velocities reachable within one time-step given the robot's current velocity and acceleration limits—the *dynamic window*—and selects the (v, ω) pair whose forward-simulated trajectory scores highest on a weighted cost function. This approach is inherently kinematic-aware and evaluates future consequences rather than reacting to the instantaneous sensor snapshot.

B. Algorithm

Algorithm 1: Dynamic Window Approach (DWA)

Input: State $(x, y, \theta, v_c, \omega_c)$, goal (g_x, g_y), LIDAR scan
Output: Command velocity (v^*, ω^*)

obs ← ScanToObstacles(scan);
if min FrontRange(*scan*) < 0.35 m **then**
  return pre-emptive avoidance command;
$[v^{dw}_{\min}, v^{dw}_{\max}, \omega^{dw}_{\min}, \omega^{dw}_{\max}] \leftarrow$ DynamicWindow(v_c, ω_c);
bestCost $\leftarrow -\infty$; $v^* \leftarrow 0$; $\omega^* \leftarrow 0$;
for $v \in [v^{dw}_{\min}, v^{dw}_{\max}]$ step Δv **do**
  for $\omega \in [\omega^{dw}_{\min}, \omega^{dw}_{\max}]$ step $\Delta\omega$ **do**
    $\tau \leftarrow$ SimulateTrajectory$(v, \omega, T_{\text{pred}})$;
    $O \leftarrow$ ObstacleCost(τ, obs);
    if $O = -\infty$ **then continue** ; `// collision--discard`
    ;
    $H \leftarrow$ HeadingCost(τ, g_x, g_y);
    $V \leftarrow |v|/v_{\max}$;
    $D \leftarrow$ DistanceCost(τ, g_x, g_y);
    $c \leftarrow w_h H + w_o O + w_v V + w_d D$;
    if $c >$ *bestCost* **then**
      bestCost $\leftarrow c$; $v^* \leftarrow v$; $\omega^* \leftarrow \omega$;
if *bestCost* $= -\infty$ **then**
  $(v^*, \omega^*) \leftarrow$ PhasedRecovery(stuckCount);
  stuckCount += 1;
return (v^*, ω^*);

C. Implementation Details

1) Dynamic Window: The velocity window clips the kinematic search to the set reachable within $\Delta t = 0.1$ s:

$$v \in \big[\max(v_{\min},\, v_c - a_{\text{lin}}\Delta t),\ \min(v_{\max},\, v_c + a_{\text{lin}}\Delta t)\big], \tag{1}$$

$$\omega \in \big[\max(-\omega_{\max},\, \omega_c - a_{\text{ang}}\Delta t),\ \min(\omega_{\max},\, \omega_c + a_{\text{ang}}\Delta t)\big]. \tag{2}$$

The window is sampled at $\Delta v = 0.02$ m/s and $\Delta\omega = 0.1$ rad/s.

2) Trajectory Simulation: Each (v, ω) sample is forward-integrated over $T_{\text{pred}} = 0.8$ s using unicycle kinematics:

$$\begin{aligned} x_{t+1} &= x_t + v\cos\theta_t\,\Delta t, \\ y_{t+1} &= y_t + v\sin\theta_t\,\Delta t, \\ \theta_{t+1} &= \theta_t + \omega\,\Delta t. \end{aligned}$$

3) Cost Function: Each trajectory is evaluated by:

$$\mathcal{C}(\tau) = w_h H + w_o O + w_v V + w_d D, \tag{3}$$

with $w_h = 1.5$, $w_o = 0.5$, $w_v = 0.3$, $w_d = 1.2$.

- H (**Heading**): angular alignment of the endpoint with the goal direction, normalised to $[0, 1]$.
- O (**Obstacle**): minimum distance from subsampled trajectory points to LIDAR obstacle points within 1.5 m. Trajectories entering a safety margin of $r_s = 0.08$ m

receive $O = -\infty$ and are discarded; otherwise $O = \min(d_{\min}/2, 1)$.

- V (**Velocity**): $|v|/v_{\max}$; rewards faster motion.
- D (**Distance**): normalised reduction in Euclidean distance to goal; penalises trajectories that diverge from goal.

4) Pre-emptive Avoidance and Recovery: If the minimum LIDAR reading within the front $\pm 30°$ arc falls below 0.35 m, a pre-emptive avoidance command ($v = -0.03$ m/s, $\omega = \pm 1.5$ rad/s toward the clearer side) bypasses the sampling loop. When all sampled trajectories are infeasible, PHASEDRECOVERY cycles through three phases (modulo 6): (1) reverse at $v = -0.05$ m/s; (2) in-place rotation at $\omega = \pm 2.0$ rad/s; (3) forward push at $v = 0.1$ m/s, $\omega = \pm 1.5$ rad/s.

D. Results

DWA publishes all sampled trajectories as a `MarkerArray` (green = safe, red = collision) and the selected trajectory in blue; the executed path is traced in cyan. The smooth S-curve path produced by DWA threads the obstacle field cleanly, consistent with zero recovery events across the entire trial, as shown in Fig. 1.

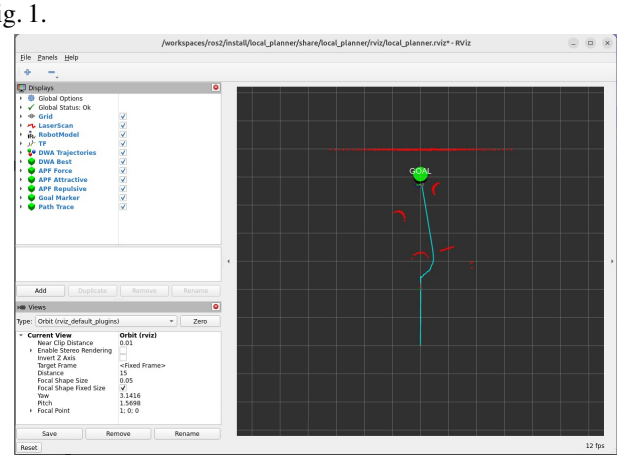

Fig. 1: DWA path trace in RViz. The smooth cyan S-curve navigates through the obstacle field to the GOAL marker without triggering any recovery behaviour.

The velocity-space trajectory fan around the dynamic obstacles is shown in Fig. 2; the fan visibly re-orients each cycle as obstacles move, demonstrating real-time reactivity.

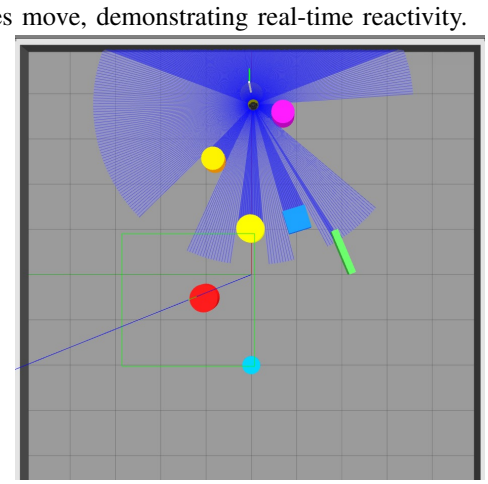

Fig. 2: DWA trajectory fan in Gazebo during the dynamic-obstacle trial. Blue fans show candidate trajectories sampled each cycle; coloured spheres are moving obstacles.

As recorded in Fig. 3, DWA reached the goal in **36.3 s** covering **6.28 m** with a minimum obstacle clearance of **0.08 m** and **zero** recovery events.

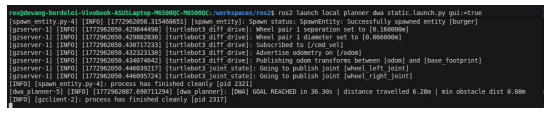

Fig. 3: DWA terminal output confirming goal reached in 36.3 s, path length 6.28 m, minimum obstacle clearance 0.08 m, zero recovery events.

V. ARTIFICIAL POTENTIAL FIELD

A. Overview

The Artificial Potential Field method [2] drives the robot along the negative gradient of a superimposed virtual potential: the goal exerts an attractive potential drawing the robot forward, while each obstacle within influence radius D_0 contributes a repulsive potential. The principal limitation of classical APF is susceptibility to *local minima* [2], where the attractive and repulsive force vectors cancel, causing the robot to freeze indefinitely. Three complementary mechanisms are implemented to mitigate this: tangential force blending, direction memory, and gap-based escape.

B. Algorithm

Algorithm 2: REPULSIVEWITHTANGENTIAL

Input: LIDAR scan, $\mathbf{F}_{\text{att}}$
Output: Blended force $\mathbf{F}_{\text{out}}$

$\mathbf{F}_{\text{rep}} \leftarrow \mathbf{0}$;
foreach *LIDAR point i with $d_i < D_0$=0.8 m* **do**
  $\mathbf{F}_{\text{rep}} \mathrel{+}= K_{\text{rep}} \left(\frac{1}{d_i} - \frac{1}{D_0} \right) \frac{\hat{\mathbf{d}}_i}{d_i^2}$;
if $\|\mathbf{F}_{\text{rep}}\| < \varepsilon$ **then return** $\mathbf{0}$;
$\mathbf{F}^{+} \leftarrow \text{rotate}(\mathbf{F}_{\text{rep}}, +90°)$;
$\mathbf{F}^{-} \leftarrow \text{rotate}(\mathbf{F}_{\text{rep}}, -90°)$;
if `tangHoldCnt`>0 **then**
  Use committed `tangDir`;
  `tangHoldCnt` $-= 1$;
else
  `tangDir` $\leftarrow \arg\max_{\pm}(\mathbf{F}^{\pm} \cdot \mathbf{F}_{\text{att}})$;
  `tangHoldCnt` $\leftarrow 40$; `// hold direction` $\approx$4 s
$\text{opp} \leftarrow \max(0, -\cos\angle(\mathbf{F}_{\text{att}}, \mathbf{F}_{\text{rep}}))$;
$\alpha \leftarrow 0.30 + 0.70 \cdot \text{opp}$;
return $(1-\alpha)\mathbf{F}_{\text{rep}} + \alpha K_T \mathbf{F}_{\text{tang}}$;

Algorithm 3: APF with Local-Minima Escape

Input: Pose (x, y, θ), goal (g_x, g_y), LIDAR scan
Output: Command velocity (v^*, ω^*)

```
if escapeTicks > 0 then
    if escapeTicks > 20 then
        return v*=−0.12 m/s,;
        ω*=0 ;          // back-up phase (1 s)
    else
        return v*=0.10 m/s,;
        ω*=escapeAng ;     // gap-steer phase (2 s)
F_att ← ATTRACTIVE(x, y, g_x, g_y);
F_rep ← REPULSIVEWITHTANGENTIAL(scan, F_att);
F_tot ← F_att + F_rep;
if CHECKSTUCK(posHistory) then
    stuckCount += 1;   escapeTicks ← 30;
    φ ← FINDGAPANGLE(scan, g_x, g_y);
    escapeAng ← clip(2.5 · angErr(φ), −1.8, 1.8);
    Flip tangDir;   clear posHistory;
    return;
return FORCETOCMD(F_tot);
```

C. Implementation Details

1) Attractive Force: A conic-quadratic hybrid potential avoids unbounded force at large distances while providing smooth deceleration on approach:

$$\mathbf{F}_{\text{att}} = \begin{cases} K_{\text{att}}(\mathbf{g} - \mathbf{p}), & d \le 1.0\ \text{m}, \\ K_{\text{att}} \dfrac{\mathbf{g} - \mathbf{p}}{d}, & d > 1.0\ \text{m}, \end{cases} \tag{4}$$

with $K_{\text{att}} = 1.2$ and $d = \|\mathbf{g} - \mathbf{p}\|$.

2) Repulsive and Tangential Forces: Repulsion follows the classical Khatib formulation summed over all LIDAR returns within $D_0 = 0.8\,\text{m}$:

$$\mathbf{F}_{\text{rep}} = K_{\text{rep}} \sum_i \left(\frac{1}{d_i} - \frac{1}{D_0} \right) \frac{\hat{\mathbf{d}}_i}{d_i^2}, \tag{5}$$

with $K_{\text{rep}} = 0.5$. When $\mathbf{F}_{\text{rep}}$ directly opposes $\mathbf{F}_{\text{att}}$ (local-minimum condition), the repulsive vector is rotated 90° to form a tangential bypass force and blended in:

$$\mathbf{F}_{\text{out}} = (1 - \alpha)\mathbf{F}_{\text{rep}} + \alpha K_T \mathbf{F}_{\text{tang}}, \tag{6}$$

where $\alpha = 0.30 + 0.70 \cdot \text{opp}$, $K_T = 1.2$, and $\text{opp} = \max(0, -\cos\angle(\mathbf{F}_{\text{att}}, \mathbf{F}_{\text{rep}}))$. A *direction memory* of 40 ticks ($\approx$4 s) prevents oscillation between clockwise and counter-clockwise bypass once a direction is committed.

3) Gap-Based Escape: If the robot displaces less than 5 cm over a 25-tick sliding window ($\approx$2.5 s), FINDGAPANGLE identifies the free LIDAR ray ($r > 0.45\,\text{m}$) whose angle is closest to the goal direction. A 30-tick escape sequence then executes: 10 ticks ($\approx$1 s) of reverse at $-0.12\,\text{m/s}$, followed by 20 ticks ($\approx$2 s) of forward steering toward the gap at $\omega = \pm 1.8\,\text{rad/s}$. The tangential bypass direction is flipped after each escape to prevent repeating the same failed manoeuvre.

D. Results

APF publishes three arrow markers per cycle—attractive (blue), repulsive (red), resultant (green)—and traces the executed path in cyan. The characteristic oscillatory looping behaviour arising from repeated local-minimum encounters at the same obstacle cluster is shown in Fig. 4.

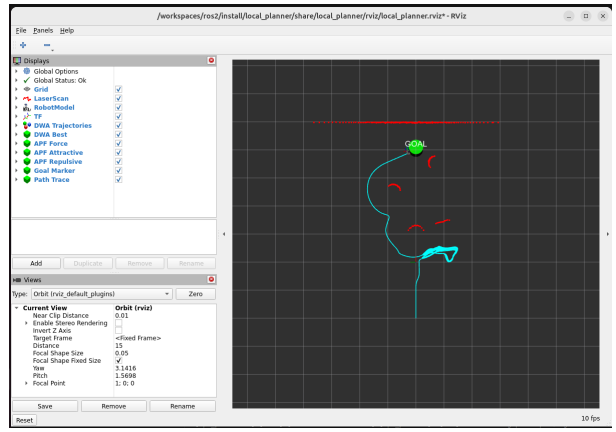

Fig. 4: APF path trace in RViz. The oscillatory cyan trace reveals repeated looping around the same obstacle cluster, in stark contrast to the smooth DWA trajectory in Fig. 1.

The force-vector visualisation and the notably wider sweeping manoeuvres required to navigate the same environment that DWA traversed directly are visible in Fig. 5.

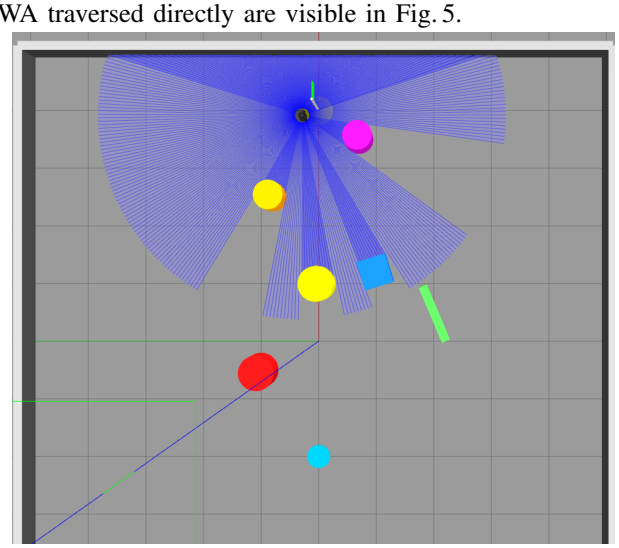

Fig. 5: APF in Gazebo. The wider sweeping manoeuvres and force-field arrow markers (blue = attractive, red = repulsive) are visible around the obstacle cluster.

As shown in Fig. 6, APF triggered the gap-escape recovery **11 consecutive times**, all at the same gap angle (-1.80 rad), indicating persistent failure of the tangential force mechanism at one specific local minimum. The goal was ultimately reached in **388 s** over **70.77 m**.

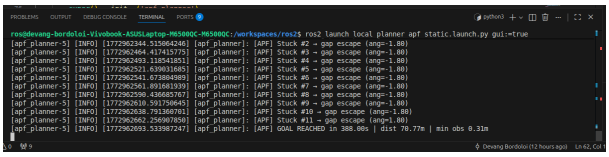

Fig. 6: APF terminal output: 11 consecutive gap-escape events triggered at gap angle -1.80 rad; goal reached in 388 s over 70.77 m.

VI. Dynamic Obstacle Extension

To evaluate planner robustness beyond static environments, a dedicated node `dynamic_obstacle_manager.py` was developed. It spawns three SDF box obstacles in Gazebo via the `/spawn_entity` service at startup, then relocates them at 20 Hz via `/set_entity_state`, implementing linear back-and-forth motion along predefined paths. Table I lists the obstacle parameters; the node probes both `/set_entity_state` and `/gazebo/set_entity_state` at initialisation for compatibility across Gazebo versions.

TABLE I: Dynamic Obstacle Parameters

Name	Size (m)	Path	Speed
dyn_obs_1	$0.30\times0.30\times0.50$	$x{=}0,\ y{\in}[-2,2]$	0.25 m/s
dyn_obs_2	$0.40\times0.40\times0.50$	$x{=}2,\ y{\in}[-2,2]$	0.20 m/s
dyn_obs_3	$0.35\times0.35\times0.50$	$x{=}3,\ y{\in}[-1.5,1.5]$	0.30 m/s

Both planners re-evaluate the environment from fresh LIDAR scans every 0.1 s, conferring inherent reactivity to moving obstacles. DWA's 0.8 s prediction horizon additionally allows it to anticipate that maintaining a current velocity will collide with an obstacle's *future* position, enabling preemptive braking or lateral swerves visible as the trajectory fan re-orientating in Fig. 2. APF responds to the obstacle's *instantaneous* position only; rapid obstacle motion therefore produces a delayed repulsive force update, generating abrupt avoidance corrections and near-miss events consistent with the higher stuck count observed even in the static trial.

VII. Experimental Results

Table II summarises the quantitative metrics recorded from the actual Gazebo runs on the identical environment.

TABLE II: Experimental Results — DWA vs. APF

Metric	DWA	APF
Goal Reached	Yes	Yes (eventually)
Time to Goal (s)	36.3	388.0
Distance Travelled (m)	6.28	70.77
Min. Obstacle Dist. (m)	0.08	0.31
Recovery Events	0	11 (gap escape)
Path Character	Smooth S-curve	Oscillatory loops
Time Ratio (APF/DWA)	$10.7\times$	
Distance Ratio (APF/DWA)	$11.3\times$	

The time-to-goal ratio of $388/36.3 \approx 10.7\times$ and path-length ratio of $70.77/6.28 \approx 11.3\times$ quantify the extent to which APF's local-minimum encounters degraded efficiency. Notably, the larger minimum obstacle clearance for APF (0.31 m vs. 0.08 m for DWA) does not indicate superior safety; rather, it reflects the robot circumnavigating obstacles at distance due to looping behaviour, not navigating past them efficiently. DWA's 0.08 m clearance is precisely its configured safety threshold r_s, confirming tight but safe navigation.

VIII. Discussion

A. Comparative Analysis

Table III presents a structured qualitative comparison across five dimensions relevant to practical deployment.

TABLE III: Qualitative Comparison of DWA and APF

Criterion	DWA	APF
Safety	Excellent: collision checked over full 0.8 s horizon per trajectory	Moderate: purely reactive, no temporal look-ahead
Path Quality	High: kinematic window ensures smooth, bounded velocity changes	Low: instantaneous force sum may produce jerky commands
Local Minima	Robust: simultaneous evaluation of all admissible trajectories eliminates single-point traps	Vulnerable: requires custom tangential and gap-escape heuristics
Computation	$\mathcal{O}(N_v \cdot N_\omega \cdot T_{\text{pred}}/\Delta t)$ per cycle; heavier but parallelisable	$\mathcal{O}(N_{\text{scan}})$ per cycle; lightweight
Dynamic Env.	Strong: trajectory horizon implicitly considers obstacle kinematics	Weak: instantaneous snapshot causes delayed response

B. Why DWA Outperforms APF

DWA's decisive advantage stems from its *proactive* architecture. By simultaneously evaluating dozens of candidate trajectories over a 0.8 s horizon and rejecting any that violate the safety margin, DWA avoids traps before they form. This is evidenced by the zero stuck-event count across the entire trial.

APF's fundamental weakness is the instantaneous, single-vector nature of its force computation. When attractive and repulsive forces cancel at a local minimum, the robot freezes; no amount of gain tuning eliminates this for arbitrary environments without additional escape logic. The three-layer escape system implemented here (tangential blending, direction memory, gap-based escape) delayed goal arrival by a factor of $10.7\times$ and increased path length by $11.3\times$, with 11 individual escape events all at the same geometric location.

From a path-quality perspective, DWA's dynamic window explicitly limits velocity change rates, producing smooth, actuator-friendly motion. APF's instantaneous force summation can generate large discontinuous velocity commands when obstacles suddenly enter or leave the influence radius, stressing real robot actuators and degrading odometry.

C. Limitations

This study has several limitations that should be acknowledged. First, all experiments were conducted in simulation; real-world

factors such as sensor noise, wheel slip, and communication latency may affect both planners differently. Second, APF's performance is highly sensitive to gain selection (K_{att}, K_{rep}, D_0); a more extensive hyperparameter search might narrow the performance gap, though it cannot eliminate the fundamental local-minimum vulnerability. Third, DWA's computational cost scales with the resolution of the velocity grid; on heavily resource-constrained hardware its real-time guarantees may not hold without algorithmic optimisation or subsampling. Finally, only one trial per planner was run per environment configuration; statistical averaging over multiple runs would provide more robust quantitative conclusions.

IX. Conclusion

This paper presented a rigorous implementation and evaluation of two classical reactive local motion planning algorithms—DWA and APF—on a TurtleBot3 Burger in Gazebo under ROS2, extended to include a dynamic obstacle environment. Complete pseudocode, mathematical formulations, and real experimental metrics were provided for both planners.

The experimental results demonstrate a clear and quantifiable performance hierarchy: **DWA outperforms APF by** $10.7\times$ **in time-to-goal and** $11.3\times$ **in path length** on an identical trial, with zero recovery events against eleven for APF. The root cause is architectural: DWA's forward-looking trajectory evaluation proactively avoids local minima, while APF's reactive force-field paradigm requires costly heuristic recovery that nonetheless fails repeatedly at the same geometric configuration.

These findings suggest that for deployment in cluttered or dynamic real-world environments, **DWA is the recommended approach**. Future work should investigate hybrid methods—using the computational efficiency of APF for coarse force guidance combined with a lightweight trajectory check to detect and prevent local-minimum conditions before they cause freezing—potentially achieving the best of both paradigms.

References

[1] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, Mar. 1997.

[2] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

[3] J. Borenstein and Y. Koren, "Real-time obstacle avoidance for fast mobile robots in cluttered environments," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 21, no. 5, pp. 1179–1187, 1991.

[4] S. S. Ge and Y. J. Cui, "New potential functions for mobile robot path planning," *IEEE Trans. Robotics and Automation*, vol. 16, no. 5, pp. 615–620, Oct. 2000.

[5] R. Simmons, "The curvature-velocity method for local obstacle avoidance," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Minneapolis, MN, 1996, pp. 3375–3382.

[6] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Efficient trajectory optimization using a sparse model," in *Proc. European Conf. Mobile Robots (ECMR)*, Barcelona, Spain, 2013, pp. 138–143.

[7] ROBOTIS, "TurtleBot3 Burger Technical Specifications," ROBOTIS e-Manual, 2023. [Online]. Available: https://emanual.robotis.com

[8] S. Macenski *et al.*, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.